In [1]:
```
import pandas as pd
import numpy as np

df = pd.read_csv("Real estate.csv")

# Clean column names
df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
    .str.replace(r"[^a-zA-Z0-9_]", "", regex=True)
)

print(df.head())
print(df.columns)
```

```
   no  x1_transaction_date  x2_house_age  \
0   1             2012.917          32.0
1   2             2012.917          19.5
2   3             2013.583          13.3
3   4             2013.500          13.3
4   5             2012.833           5.0

   x3_distance_to_the_nearest_mrt_station  x4_number_of_convenience_stores  \
0                                84.87882                               10
1                               306.59470                                9
2                               561.98450                                5
3                               561.98450                                5
4                               390.56840                                5

   x5_latitude  x6_longitude  y_house_price_of_unit_area
0     24.98298     121.54024                        37.9
1     24.98034     121.53951                        42.2
2     24.98746     121.54391                        47.3
3     24.98746     121.54391                        54.8
4     24.97937     121.54245                        43.1
Index(['no', 'x1_transaction_date', 'x2_house_age',
       'x3_distance_to_the_nearest_mrt_station',
       'x4_number_of_convenience_stores', 'x5_latitude', 'x6_longitude',
       'y_house_price_of_unit_area'],
      dtype='object')
```

In [2]:
```
print("Missing values:")
print(df.isnull().sum())

# Fill numerical missing values with median
num_cols = df.select_dtypes(include=np.number).columns

for col in num_cols:
    df[col] = df[col].fillna(df[col].median())

# Fill categorical missing values with mode
cat_cols = df.select_dtypes(include="object").columns

for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])
```

```
print("\nMissing values after cleaning:")
print(df.isnull().sum())
```

```
Missing values:
no                                      0
x1_transaction_date                     0
x2_house_age                            0
x3_distance_to_the_nearest_mrt_station    0
x4_number_of_convenience_stores         0
x5_latitude                             0
x6_longitude                            0
y_house_price_of_unit_area              0
dtype: int64

Missing values after cleaning:
no                                      0
x1_transaction_date                     0
x2_house_age                            0
x3_distance_to_the_nearest_mrt_station    0
x4_number_of_convenience_stores         0
x5_latitude                             0
x6_longitude                            0
y_house_price_of_unit_area              0
dtype: int64
```

In [3]:
```
# Example: merge with another dataset if available
# additional_df = pd.read_csv("Neighborhood_Data.csv")

# df = pd.merge(
#     df,
#     additional_df,
#     on="neighborhood",
#     how="left"
# )

print("Data merging can be performed when an additional dataset is availa
```

```
Data merging can be performed when an additional dataset is available.
```

In [4]:
```
# Display numerical columns
print(df.select_dtypes(include=np.number).columns)

# Example: filter properties with price greater than 1,000,000
# Change the column name according to your dataset

if "house_price_of_unit_area" in df.columns:
    filtered_data = df[df["house_price_of_unit_area"] > 30]
    print(filtered_data.head())
```

```
Index(['no', 'x1_transaction_date', 'x2_house_age',
       'x3_distance_to_the_nearest_mrt_station',
       'x4_number_of_convenience_stores', 'x5_latitude', 'x6_longitude',
       'y_house_price_of_unit_area'],
      dtype='object')
```

In [5]:
```
# Find categorical columns
categorical_columns = df.select_dtypes(include="object").columns

print("Categorical columns:")
print(categorical_columns)
```

```
# One-hot encoding
df_encoded = pd.get_dummies(
    df,
    columns=categorical_columns,
    drop_first=True
)

print(df_encoded.head())
```

```
Categorical columns:
Index([], dtype='object')
   no  x1_transaction_date  x2_house_age  \
0   1             2012.917          32.0
1   2             2012.917          19.5
2   3             2013.583          13.3
3   4             2013.500          13.3
4   5             2012.833           5.0

   x3_distance_to_the_nearest_mrt_station  x4_number_of_convenience_stores  \
0                                84.87882                               10
1                               306.59470                                9
2                               561.98450                                5
3                               561.98450                                5
4                               390.56840                                5

   x5_latitude  x6_longitude  y_house_price_of_unit_area
0     24.98298     121.54024                        37.9
1     24.98034     121.53951                        42.2
2     24.98746     121.54391                        47.3
3     24.98746     121.54391                        54.8
4     24.97937     121.54245                        43.1
```

In [6]:
```
# Display average price
if "house_price_of_unit_area" in df.columns:
    print("Average House Price:")
    print(df["house_price_of_unit_area"].mean())

# Example aggregation by a categorical column
# Replace 'location' with the appropriate column if available

# df.groupby("location")["house_price_of_unit_area"].mean()
```

In [7]:
```
# Detect outliers using IQR
numeric_columns = df.select_dtypes(include=np.number).columns

for col in numeric_columns:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    outliers = df[(df[col] < lower) | (df[col] > upper)]

    print(col, ":", len(outliers), "outliers")
```

```
no : 0 outliers
x1_transaction_date : 0 outliers
x2_house_age : 0 outliers
x3_distance_to_the_nearest_mrt_station : 37 outliers
x4_number_of_convenience_stores : 0 outliers
x5_latitude : 8 outliers
x6_longitude : 35 outliers
y_house_price_of_unit_area : 3 outliers
```

In [8]:
```python
for col in numeric_columns:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    # Cap extreme values
    df[col] = df[col].clip(lower, upper)

print("Outliers handled successfully.")
```

```
Outliers handled successfully.
```